

Deliverable 1 ? Security Planning Report

Course: CY321 ? Secure Software Design with Secure Coding Implementation

Deliverable: D1 ? Security Planning Phase

Project: AUI Secure ? DevSecOps CI/CD Pipeline & Secure Coding Demonstration Platform

Date: 2026-05-07

1. What Deliverable 1 Required

Deliverable 1 covers the security planning phase of the project ? everything that happens before you write a single line of code. The required outputs were:

- Project Proposal ? What we are building and why
- Security Requirements Planning ? What security properties the system must have
- Risk Management ? What could go wrong and how bad it would be
- Threat Modeling ? Specific threats against our system and how we will counter them

The documents produced are in Deliverable-1/:

- Project_Proposal.pdf
- D1_Security_Requirements_Planning.pdf
- D1_Risk_Management.pdf
- D1_Threat_Modeling.pdf

2. Project Proposal ? What We Planned to Build

2.1 The Concept

We proposed building a secure web application with an interactive security education layer. The core idea was that security is best understood by seeing attacks work in real time ? not just reading about them.

The proposed application would:

- Be a real, functional web application (not a toy)
- Demonstrate each OWASP Top-10 vulnerability live
- Show both the vulnerable version and the secure version of every feature
- Have an automated DevSecOps CI/CD pipeline that enforces security on every code push

2.2 Why This Project

Most security tools either show you the attack or show you the defence. We proposed showing both simultaneously in one platform, gated by a server-side feature flag (SECURE_MODE). This design decision ? made at the proposal stage ? became the defining architectural feature of the entire project.

2.3 How the Proposal Maps to the Final Code

Proposed Feature	Implemented As	File
Secure user authentication	scrypt hashing, session rotation, HttpOnly cookies	app/auth.py
Role-based access control	@requires_auth, @requires_role decorators	app/rbac.py

SQL injection demonstration	/lab/sqli/attack and /lab/sqli/defend endpoints	app/lab.py
XSS demonstration	/lab/xss/attack and /lab/xss/defend endpoints	app/lab.py
Security headers	7 headers on every response	app/security.py
Automated CI/CD security pipeline	5-stage GitHub Actions workflow	.github/workflows/devsecops.yml
Audit logging	log_action() called by every module	app/audit.py
Rate limiting	Flask-Limiter on auth endpoints	app/security.py

3. Security Requirements Planning ? What the System Must Do

3.1 Functional Security Requirements

These are the security behaviours the system must provide:

Requirement ID	Requirement	Priority	Implementation Status
SR-01	Passwords must never be stored in plain text	Critical	DONE ? script hashing in auth.py
SR-02	Failed login attempts must trigger account lockout	Critical	DONE ? exponential backoff in auth.py
SR-03	Users must only access their own data	Critical	DONE ? WHERE user_id = ? in items.py
SR-04	Admin routes must be restricted to admin role	Critical	DONE ? @requires_role("admin") in rbac.py
SR-05	All user input must be validated before use	Critical	DONE ? validators.py applied at every endpoint
SR-06	SQL queries must use parameterised statements	Critical	DONE ? all secure-path queries use ? placeholders
SR-07	Session cookies must use HttpOnly and SameSite flags	High	DONE ? configured in config.py and main.py
SR-08	CSRF tokens must protect state-changing requests	High	DONE ? csrf_protect decorator in security.py
SR-09	Security headers must be sent with every response	High	DONE ? _apply_security_headers in security.py
SR-10	Rate limiting must be applied to authentication endpoints	High	DONE ? Flask-Limiter in auth.py
SR-11	All security-relevant actions must be logged	Medium	DONE ? audit.py called everywhere
SR-12	Dependencies must be scanned for known CVEs	Medium	DONE ? pip-audit in CI pipeline Stage 4
SR-13	Source code must be scanned for hardcoded secrets	Medium	DONE ? gitleaks in CI pipeline Stage 2
SR-14	API keys must be stored as hashes, not plain text	Medium	DONE ? SHA-256 hashing in api_keys.py

3.2 Non-Functional Security Requirements

Requirement	Target	Achieved
Availability	Audit logging must never crash the main application	Yes ? try/except swallows audit errors
Maintainability	All security configuration from environment	Yes ? config.py reads all settings from os.environ
Testability	Security tests must cover all OWASP Top-10 categories	Yes ? 40 security tests in test_security.py
Deployability	App must be safe by default with zero	Yes ? SECURE_MODE=true by default
Auditability	Every action must be traceable to a user, IP, and timestamp	Yes ? audit_logs table with full metadata

4. Risk Management ? What Could Go Wrong

4.1 Risk Identification and Assessment

The risk management phase identified the following risks (likelihood × impact = risk score):

Risk ID	Risk Description	Category	Likelihood	Impact	Score	Rating
R-01	SQL injection dumps user	Technical	5	5	25	CRITICAL
R-02	Brute-force attack compromises user accounts	Technical	4	5	20	HIGH
R-03	XSS attack steals session cookies	Technical	4	4	16	HIGH
R-04	CSRF attack performs actions on behalf of logged-in	Technical	4	4	16	HIGH
R-05	IDOR allows users to access other users' data	Technical	5	4	20	HIGH
R-06	Privilege escalation from user to admin	Technical	3	5	15	HIGH
R-07	Hardcoded secret committed to git	Process	3	5	15	HIGH
R-08	Vulnerable dependency exploited via known	Technical	3	4	12	MEDIUM
R-09	Missing security headers allow	Technical	3	3	9	MEDIUM
R-10	Session fixation after login	Technical	2	5	10	MEDIUM
R-11	Timing attack reveals which					

usernames exist

Technical	3	2	6	LOW
-----------	---	---	---	-----

R-12	Worker thread DoS via sleep in request handler	Technical	3	3	9	MEDIUM
------	--	-----------	---	---	---	--------

4.2 Risk Treatment Plan

Risk ID	Treatment Strategy	Control Implemented	Residual Risk
R-01	Mitigate	Parameterised queries everywhere; SQLi blocklist as extra layer	Low
R-02	Mitigate	Account lockout with exponential backoff; rate limiting 10/min	Low
R-03	Mitigate	bleach + html.escape + CSP nonce; HttpOnly cookies	Low
R-04	Mitigate	256-bit CSRF token + SameSite=Lax cookie	Low
R-05	Mitigate	WHERE user_id = ? using session-bound ID; SECURE_MODE	Low
R-06	Mitigate	@requires_role reads role from session (server-side only); role	Low
R-07	Mitigate	gitleaks pre-commit hook + CI scan; RuntimeWarning if SECRET_KEY	Low
R-08	Mitigate	pip-audit in CI + auto-remediation PR	Low
R-09	Mitigate	7 security headers applied to every response in after_request hook	Low
R-10	Mitigate	session.clear() before writing any data on login	Low
R-11	Mitigate	Dummy hash check on unknown users equalises response time	Very Low
R-12	Mitigate (D3)	time.sleep() removed; replaced with timing-safe dummy hash	Low

4.3 Risk Residual Assessment

After applying all controls, the residual risk profile is:

- Critical risks remaining: 0
- High risks remaining: 0
- Medium risks remaining: 1 (R-12 ? partially addressed, monitoring recommended)
- Low/Very Low risks remaining: All others

5. Threat Modeling ? STRIDE Analysis

The STRIDE model was used to systematically identify threats across 6 categories.

5.1 Spoofing (Identity Threats)

Threat: Attacker impersonates a legitimate user or administrator.

Identified threats:

- Credential stuffing using leaked password lists
- Session fixation ? attacker sets a known session token before login
- Timing attack to enumerate valid usernames
- Cookie theft via XSS to steal session token

Controls implemented:

- script password hashing with random salt (resistant to credential stuffing)
- session.clear() on every login (prevents fixation)
- Dummy hash check for unknown users (prevents timing enumeration)
- HttpOnly cookies (prevents cookie theft via XSS)

Code: app/auth.py:login(), app/config.py:SESSION_COOKIE_HTTPONLY

5.2 Tampering (Data Integrity Threats)

Threat: Attacker modifies data in transit or injects malicious data.

Identified threats:

- SQL injection to manipulate database queries
- XSS to inject malicious scripts
- CSRF to forge authenticated requests
- Session cookie tampering to change user_id or role
- Parameter pollution (sending extra fields like role: admin in JSON)

Controls implemented:

- Parameterised queries with ? placeholders (SQL injection)
- bleach + html.escape + CSP (XSS)
- CSRF token + SameSite=Lax (CSRF)
- HMAC-signed sessions via itsdangerous (tampering)
- Only explicitly whitelisted fields accepted from request body

Code: app/validators.py, app/security.py:csrf_protect, app/security.py:Content-Security-Policy

5.3 Repudiation (Audit Threats)

Threat: User denies having performed an action; no evidence exists.

Identified threats:

- No record of login attempts or failures
- No record of data modifications
- No record of admin privilege use

Controls implemented:

- audit.log_action() called on every state-changing action
- Records user_id, username, IP address, User-Agent, action, success, timestamp
- Append-only table ? application never deletes audit records

Code: app/audit.py:log_action() ? called in auth.py, items.py, lab.py, dashboard.py, rbac.py

5.4 Information Disclosure (Confidentiality Threats)

Threat: Sensitive data is exposed to unauthorised parties.

Identified threats:

- IDOR ? changing a URL ID to access another user's data
- SQL injection dumping the users table including password hashes
- Error messages leaking stack traces or file paths
- Session token leaking via Referer header

Controls implemented:

- WHERE user_id = ? scopes every query (IDOR)
- Parameterised queries (SQL injection)
- Generic JSON error handlers with no stack traces (error leakage)
- Referrer-Policy: strict-origin-when-cross-origin (Referer leakage)

Code: app/items.py:_uid(), app/main.py:server_error(), app/security.py:Referrer-Policy

5.5 Denial of Service (Availability Threats)

Threat: Attacker prevents legitimate users from using the service.

Identified threats:

- Login flood exhausting server resources
- time.sleep() in login handler blocking worker threads
- Oversized payloads consuming memory
- Lockout-based DoS (attacker locks out legitimate users)

Controls implemented:

- Flask-Limiter: 10/min on auth, 200/day and 50/hour globally (rate limiting)
- time.sleep() removed from login handler (DoS amplifier fix ? D3 F2)
- Length caps: username 32, password 128, title 200, body 10000, search 100
- Lockout resets on successful login (prevents weaponised lockout)

Code: app/security.py:limiter, app/auth.py:_record_failure, app/validators.py

5.6 Elevation of Privilege (Access Control Threats)

Threat: User gains more privileges than they are entitled to.

Identified threats:

- Regular user accessing admin dashboard
- Any user disabling SECURE_MODE to expose attack endpoints
- Attacker sending role: admin in registration body
- SQL injection to modify role column in database

Controls implemented:

- @requires_role("admin") on all admin routes
- SECURE_MODE toggle changed to @requires_role("admin") ? D3 fix F3
- Registration always sets role='user', never from request body
- Parameterised queries prevent any SQL injection

Code: app/rbac.py:requires_role, app/lab.py:toggle_secure_mode, app/auth.py:register

6. How D1 Planning Shaped the Final Code

Every security control in the final application traces back to a risk or threat identified in Deliverable 1:

D1 Finding	Code That Addresses It	Result
R-01 SQL injection is critical	Parameterised queries throughout	6 SQLi payloads all blocked in tests
R-02 Brute force is high risk	account_lockouts table + exponential backoff	Lockout verified in test_account_lockout_after_max_attempts
R-03 XSS is high risk	bleach + html.escape + CSP nonce	3 XSS payloads blocked in tests
R-04 CSRF is high risk	csrf_protect decorator + SameSite cookie	CSRF token verified in test_csrf_token_returned_on_login
R-05 IDOR is high risk	WHERE user_id = ?	test_delete_other_users_item_fails passes
R-06 Privilege escalation	@requires_role reads server-side session	test_non_admin_dashboard_blocked passes
R-07 Hardcoded secrets	gitleaks in CI	0 secrets found in repository history
R-08 Vulnerable deps	pip-audit + auto-remediation	0 CVEs found in 57 packages
R-12 Sleep DoS (found in D3 review)	time.sleep() removed	F2 fix applied and verified

7. Summary

Deliverable 1 established the security contract for the entire project. Every requirement became a test. Every risk became a control. Every threat became a decorator, a validator, or a database constraint. The planning phase was not a formality ? it directly determined what code was written and why.

D1 Verdict: All security requirements planned in D1 are implemented and verified in D3. Risk residual is Low across all identified threats.